

LFEA Topology Validator SVG Adoption Procedure

Fool-proof implementation plan for reusing the proven Topology Validator SVG architecture in the LFEA 3D piping workbench

Target repository
Advanced_Analysis

Target baseline
main @ ea33e6a...

Reference repository
XML_Compare_Uilities

Reference baseline
main @ 126df8ac...

Decision: reuse the Topology Validator's Shared Engineering SVG contracts, projection model, command-gateway pattern and interaction discipline through a new LFEA-specific editable adapter. Do not iframe the GitHub Pages application, import code from the live site at runtime, or copy the Topology Validator controller wholesale.

1. Purpose and acceptance boundary

This procedure converts the current LFEA workbench into an SVG-first engineering modelling and review surface while preserving all existing LFEA numerical authorities. The SVG layer shall author and review piping source geometry, component identity, supports, loads, code points and results. It shall not construct stiffness, derive code stress, repair topology or become the source of solver truth.

- SVG is the primary engineering authoring surface for projected XY, XZ, YZ and ISO views.
- The authoritative source is a versioned LFEA engineering model, not DOM elements or SVG path data.
- Pointer movement creates transient preview only; accepted commands update the draft/model through a revision-checked gateway.
- Solver, recovery and B31.3 packages consume immutable LFEA contracts and never inspect the SVG DOM.
- Three.js/WebGL may remain available as a read-only 3D inspection backend, but it is not required to author source geometry.

2. Reference implementation reviewed

Reference capability	Reusable principle
Native SVG workspace wiring	Topology Validator supports XY/XZ/YZ/ISO projection, fit, zoom, labels, component symbols, selection synchronization and axis compass.
TopologySvgEditorDraft.v1	A mutable draft is separated from immutable source/topology artifacts; accepted operations are exported as a patch.
Editor controller	Review/edit modes, exact coordinate entry, axis handles, transient pointer preview, one-command-per-drag history, Escape cancellation, revert and export.
Pointer and axis math	Client-to-SVG conversion, projected axis vectors and scalar drag resolution are pure functions.
EngineeringSvgAdapter.v1	Domain-neutral scene adapter with projection declarations, property provider, snap providers and optional editable command gateway.
EngineeringSvgCommandGateway.v1	Stale revision and duplicate operation IDs are rejected before model mutation.
Topology Engineering SVG adapter	The Topology domain is mapped to a shared EngineeringScene contract rather than coupling the renderer to Topology records.

2.1 Mandatory corrections during adoption

- Do not preserve Topology Validator's support-only domain restriction. Create an LFEA domain adapter for PIPE, BEND, TEE, REDUCER, RIGID, SUPPORT, LOAD and CODE_POINT entities.
- Do not preserve a read-only adapter descriptor. LFEA authoring requires `accessMode = EDITABLE` and a valid `EngineeringSvgCommandGateway.v1`.
- Do not use `localeCompare()` for canonical provider or entity ordering. Use the `Advanced_Analysis ASCII` comparator so locale cannot alter engineering identity.
- Do not use manual `viewBox`/bounding-rectangle pointer mapping without testing `preserveAspectRatio` gutters. Prefer `SVG getScreenCTM().inverse()` for screen-to-SVG conversion, with a tested fallback.
- Do not reuse Topology-specific CSS selectors, event names, evidence schemas or source IDs as LFEA authority.

3. Reuse strategy

Recommended delivery method: a source-frozen, self-contained port of the domain-neutral Engineering SVG core into Advanced_Analysis, accompanied by a provenance manifest and parity tests. A remote runtime dependency on GitHub Pages is prohibited.

Method	Disposition	Reason
--------	-------------	--------

LFEA SVG Adoption Procedure | Topology Validator Engineering SVG

Method	Disposition	Reason
Iframe or remote module from GitHub Pages	REJECT	Network-dependent, difficult to version, unsafe for engineering evidence, cross-origin lifecycle and deployment risk.
Copy Topology Validator files wholesale	REJECT	Carries Topology-specific assumptions, support-only model, CSS/event coupling and duplicate authority.
Git submodule	NOT PREFERRED	Build and checkout complexity; exact release packaging is fragile.
Published internal package	DURABLE OPTION	Best long-term sharing if release/version governance is established.
Frozen local port + provenance manifest	RECOMMENDED NOW	Self-contained builds, reviewable diff, exact source lineage, parity tests and controlled target-specific hardening.

3.1 Source provenance manifest

```
{
  "schema": "lfea-svg-upstream-provenance/v1",
  "upstreamRepository": "reallaksh19/XML_Compare_Uutilities",
  "upstreamCommit": "126df8acc370d22540cb129dce789ea04773ebaf",
  "sourceFiles": [
    {
      "path": "core/engineering-svg/engineering-svg-adapter.js",
      "blobSha": "3f912463ffb88ff3c022a5ba2a82a8eb9bd966a",
      "usage": "PORTED_AND_HARDENED"
    },
    {
      "path": "core/engineering-svg/engineering-svg-command-gateway.js",
      "blobSha": "133e852d38059a4d671f40bab1a334cf9b614478",
      "usage": "PORTED_AND_HARDENED"
    },
    {
      "path": "tabs/topology-trace-validator/topology-svg-editor-overlay.js",
      "blobSha": "9cbf0a9575ec251fca98bc466757f0a7858991e8",
      "usage": "MATH_PATTERN_REFERENCE"
    }
  ],
  "targetRepository": "reallaksh19/Advanced_Analysis",
  "targetBase": "ea33e6aacc6a3328b648468dbd6a534bc9a4c867"
}
```

4. Target architecture

LFEA engineering source / imported JSON

- > existing source validation and component graph
- > LFEA engineering-scene adapter
- > EngineeringScene.v1
- > SVG renderer + viewport state
- > selection / property / snap services
- > transient edit intent
- > revision-checked command gateway
- > LFEA draft command
- > accepted engineering-source revision
- > existing B-1 / B-2.5 / B-3.x / B-4.0 chain

Read-only optional path:

EngineeringScene.v1

- > Three.js/WebGL inspection adapter
- > orbit / perspective / dense-result review

4.1 Required target modules

Target module	Responsibility
src/workspace/lfea-svg/lfea-engineering-svg-adapter.js	Maps LFEA engineering entities to the shared EngineeringScene contract.
src/workspace/lfea-svg/lfea-svg-scene-builder.js	Creates projected semantic primitives with source IDs and result/display metadata.
src/workspace/lfea-svg/lfea-svg-command-gateway.js	Binds EngineeringSvgCommandGateway to the authoritative LFEA source revision.
src/workspace/lfea-svg/lfea-svg-draft-model.js	Mutable draft separated from accepted source; no solver imports.
src/workspace/lfea-svg/lfea-svg-history.js	Bounded undo/redo journal; one accepted command per logical edit.
src/workspace/lfea-svg/lfea-svg-overlay.js	Handles, previews, dimensions, local axes, support/load glyphs.
src/workspace/lfea-svg/lfea-svg-snap-providers.js	Endpoint, midpoint, centre, intersection, tangent and axis constraints.
src/workspace/lfea-svg/lfea-svg-properties.js	Read-only selected-entity properties and source/evidence projection.
src/workspace/lfea-svg/lfea-svg-selection.js	Point, window and crossing selection using stable entity IDs.

Target module	Responsibility
src/workspace/lfea-svg/lfea-svg-viewport.js	Projection, fit, pan, zoom and screen/world transform.
src/workspace/lfea-svg/lfea-svg-workbench.js	Lifecycle composition and state invalidation.
scripts/lfea-svg-*.mjs	Contract, parity, anti-drift, browser and performance qualification.

4.2 Contracts

Contract	Minimum meaning
LfeaSvgDraft.v1	Accepted source revision reference, draft revision, draft entities, pending commands and dirty state.
LfeaSvgCommand.v1	Operation ID, base revision, operation type, selected identities, exact before/after engineering values and actor evidence.
LfeaSvgPatch.v1	Portable ordered commands with base source hash; not directly solver-consumable.
LfeaSvgViewportState.v1	Projection, fit matrix, pan, zoom, visibility and display-only state.
LfeaSvgSelection.v1	Stable selected source/component/node IDs; no DOM-node authority.
LfeaSvgEvidence.v1	Upstream provenance, scene revision, draft revision, commands, diagnostics and parity evidence.

5. Scene model and semantic layers

Layer	Contents
construction	Grid, axis compass, guides and uncommitted construction geometry.
source	Accepted pipe centreline, component bodies and source node identities.
conditioned	Conditioned spans and analysis-node insertions, visually distinct from source.
components	Bend arcs, tee junctions, reducers, finite rigid bodies and rigid-link glyphs.
supports	Anchors, guides, line stops, springs, prescribed slots and support offsets.
loads	Forces, moments, distributed loads, temperatures and prescribed movements.
code-points	B31.3 physical evaluation stations and component-factor evidence.
results	Deformed centreline, reactions, force/moment diagrams and utilization overlays.
selection	Selected/highlighted entities and drag handles.
diagnostics	Blocking, conditional and informational entity-linked diagnostics.

5.1 Scene entity identity

- Every SVG primitive carries data-scene-entity-id and data-source-entity-id.
- DOM IDs are render-session details and never enter engineering hashes.
- One physical source component may create multiple render primitives, but every primitive maps back to exactly one source/component identity and role.
- Result overlays cite recoveryHash and codeAssessmentHash but cannot mutate either package.

6. Projection and viewport procedure

1. Implement one world-coordinate projection service supporting XY, XZ, YZ and deterministic ISO.
2. Create one fit transform from visible engineering bounds; retain it in viewport state.
3. Render all semantic layers through the same world-to-screen matrix.
4. Use SVG viewBox for zoom/pan; never scale engineering coordinates inside the source model.
5. Convert pointer coordinates with getScreenCTM().inverse(); reject a null or singular CTM.
6. Retain the manual viewBox mapping only as a tested fallback for constrained browser environments.
7. Projection or display changes do not invalidate engineering packages.

```
export function clientToSvgPoint(svg, clientX, clientY) {
  const matrix = svg.getScreenCTM();
  if (!matrix) {
    throw new Error('LFEA_SVG_SCREEN_MATRIX_UNAVAILABLE');
  }

  const determinant =
    matrix.a * matrix.d - matrix.b * matrix.c;

  if (!Number.isFinite(determinant) || determinant === 0) {
    throw new Error('LFEA_SVG_SCREEN_MATRIX_SINGULAR');
  }
}
```

```

return new DOMPoint(clientX, clientY)
  .matrixTransform(matrix.inverse());
}

```

6.1 3D editing discipline in projected SVG

- A free pointer drag may change only coordinates visible in the active projection.
- Movement along the hidden axis requires an explicit axis handle, typed coordinate or synchronized second view.
- ISO drag is axis-constrained; free-plane ISO movement is prohibited for engineering commits.
- Topology-changing operations show a preview and require explicit acceptance.
- Any edit that cannot be represented uniquely in the active projection is BLOCKED with a recommendation to switch projection or enter dimensions.

7. Command and draft lifecycle

```

pointerdown
-> capture pointer
-> record base source revision
-> create transient preview state

pointermove
-> pure projected constraint solve
-> update preview SVG only
-> no model mutation
-> no hash change

pointerup
-> construct LfeaSvgCommand.v1
-> validate current base revision
-> validate component-specific semantics
-> execute through EngineeringSvgCommandGateway.v1
-> append one history command
-> rebuild accepted scene
-> invalidate downstream solver/result packages

Escape / pointercancel
-> discard preview
-> restore accepted scene
-> no command and no revision change

```

7.1 Editable adapter skeleton

```

export function createLfeaEngineeringSvgAdapter({
  getScene,
  subscribeScene,
  getRevision,
  executeCommand,
  propertyProvider,
  snapProviders,
}) {
  const commandGateway = createEngineeringSvgCommandGateway({
    getCurrentRevision: getRevision,
    execute: executeCommand,
  });

  return createEngineeringSvgAdapter({
    descriptor: {
      adapterId: 'LFEA-ENGINEERING-SVG-R1',
      domainType: 'LFEA_PIPE_STRESS',
      accessMode: 'editable',
      projections: ['ISO', 'XY', 'XZ', 'YZ'],
      capabilities: {
        selection: true,
        properties: true,
        parity: false,
        editing: true,
      },
      metadata: {
        owner: 'lfea-workbench',
      },
    },
    getScene,
    subscribeScene,
    propertyProvider,
    snapProviders,
    commandGateway,
  });
}

```

8. Component-specific editing rules

Entity	Permitted and prohibited behavior
PIPE	Move endpoint, split span, change endpoint coordinate, assign section/material. A zero-length result blocks. Intermediate support/load/code stations are preserved or explicitly remapped.
BEND	Edit tangent points, centre/radius/sweep or declared arc authority. Free polyline deformation is prohibited. Commit must re-pass bend geometry and B-3.2 subdivision/flexibility checks.
TEE	Move the explicit junction or leg endpoints. The UI may preview leg roles, but direction-vector topology remains the run/branch classifier.
REDUCER	Edit length, end stations and section mapping. Visual taper is derived; the stepped approximation and CONDITIONAL state remain visible.
RIGID finite body	Edit endpoints, length and CG through exact fields/handles. Zero length blocks unless the governed weight-lump rule applies.
RIGID link	Edit master, slave and offset; code-stress eligibility remains false and cannot be changed in the SVG UI.
SUPPORT	Move support location/offset, edit direction/stiffness through properties. One-way/gap/friction semantics remain outside the linear release.
LOAD	Move an application station only through an accepted source edit; visual arrow scaling is display-only.

9. Snapping and constraints

Snap mode	Authority
ENDPOINT	Canonical node or component port.
MIDPOINT	Pipe span midpoint; creates a preview station, not an implicit node until accepted.
CENTRE	Bend centre or circular component centre.
INTERSECTION	Projected intersection; 3D compatibility must be proven before commit.
TANGENT	Bend tangent geometry under the active projection.
PERPENDICULAR	Constraint to a selected reference span.
AXIS	Global X, Y or Z axis-constrained movement.
GRID	Display convenience only; accepted coordinate is the exact source value.

9.1 Snap selection rules

- Snap aperture is a display profile expressed in screen pixels and does not enter the engineering hash.
- Candidate IDs are deterministic and ASCII sorted.
- Equal-distance ties use declared provider priority followed by candidate ID.
- A projected intersection that represents skew 3D lines is visible as NON_COMMITTABLE.
- The committed engineering point is recalculated and validated in world coordinates.

10. Result overlays

- Deformation is display-only and requires an explicit positive scale.
- Force and moment diagrams consume B-3.4 recovery values and retain local/global basis labels.
- B31.3 utilization consumes B-4.0 code results and retains category, code point, combination and source dataset identity.
- The workspace must not recompute stress, SIF, allowable or utilization.
- A stale result package is not rendered as current and is not exportable.

11. Phased implementation procedure

Work pack	Scope	Exit
LFEA-SVG-0	Source freeze and parity inventory	Record upstream commit/blob SHAs; identify domain-neutral modules and target corrections; run current target gates.
LFEA-SVG-1	Core contracts and read-only scene	Port shared scene/adaptor essentials; create LFEA read-only scene adaptor and projection parity tests.
LFEA-SVG-2	Viewport, navigation and selection	XY/XZ/YZ/ISO, fit/pan/zoom, compass, stable entity selection, property inspector and accessibility.
LFEA-SVG-3	Draft, command gateway and history	Transient preview, stale-revision rejection, one-command-per-drag, undo/redo, revert and patch evidence.
LFEA-SVG-4	Pipe and node editing	Move/split/edit pipe spans, typed coordinates and station preservation.

Work pack	Scope	Exit
LFEA-SVG-5	Piping components	Bend, tee, reducer and rigid authoring integrated with B-3.2 ownership and validation.
LFEA-SVG-6	Supports, loads and code points	Symbols, property edits, application stations and downstream invalidation.
LFEA-SVG-7	Results and optional WebGL inspection	Deformation, diagrams, reactions, utilization and synchronized read-only 3D review.
LFEA-SVG-8	Performance, E2E and release	Large-model bands, accessibility, deterministic export, anti-drift and exact-head qualification.

12. Test and benchmark plan

Test group	Required proof
Projection	Known-point XY/XZ/YZ/ISO values; fit bounds; pan/zoom round trip; resize; devicePixelRatio; preserveAspectRatio gutters.
Pointer math	Client-to-SVG parity, nested transforms, axis vector, drag scalar, hidden-axis blocking and pointer capture.
Revision safety	Stale base revision, duplicate operation ID, concurrent edit, rejected command and failure cleanup.
Immutability	Preview cannot mutate source; cancel leaves bytes/hash unchanged; source and solver packages deeply frozen.
History	One command per drag; undo/redo; redo truncation after new command; revert; bounded history.
Selection	Click, window, crossing, hidden/isolate layers, stable IDs and synchronized table/viewport selection.
Components	PIPE, BEND, TEE, REDUCER, finite RIGID and kinematic RIGID valid/invalid cases.
Downstream	Accepted edit invalidates topology/model/execution/recovery/code/report; display change invalidates none.
Results	Deformation scale, diagram basis, stale result blocking and B31.3 trace action.
Accessibility	Keyboard selection, focus retention, labelled controls, visible focus and equivalent result table.
Performance	1k, 10k and 50k display entities; pan/zoom latency; selection latency; DOM count; memory.

12.1 Mandatory named tests

- LFEA-SVG-T01 upstream provenance manifest exact
- LFEA-SVG-T02 read-only scene parity with Topology Validator fixture
- LFEA-SVG-T03 ASCII ordering independent of locale
- LFEA-SVG-T04 getScreenCTM pointer mapping under aspect-ratio gutters
- LFEA-SVG-T05 XY/XZ/YZ/ISO projection benchmark
- LFEA-SVG-T06 axis drag X/Y/Z in every projection
- LFEA-SVG-T07 hidden-axis free drag blocked
- LFEA-SVG-T08 preview is source-byte invariant
- LFEA-SVG-T09 Escape and pointercancel preserve revision
- LFEA-SVG-T10 stale command rejected
- LFEA-SVG-T11 duplicate operation ID rejected
- LFEA-SVG-T12 one command per completed drag
- LFEA-SVG-T13 undo/redo and redo-tail truncation
- LFEA-SVG-T14 pipe split preserves stations
- LFEA-SVG-T15 bend edit re-enters B-3.2 qualification
- LFEA-SVG-T16 tee classifier remains direction-owned
- LFEA-SVG-T17 reducer cannot flatten to PIPE
- LFEA-SVG-T18 rigid link remains code-stress ineligible
- LFEA-SVG-T19 support/load edit invalidates correct descendants
- LFEA-SVG-T20 display-only changes preserve engineering hashes
- LFEA-SVG-T21 stale recovery/code result not rendered current
- LFEA-SVG-T22 keyboard and screen-reader interaction
- LFEA-SVG-T23 10k-entity responsiveness
- LFEA-SVG-T24 deterministic patch/export bytes

12.2 Visual parity fixture

Fixture:
 one straight PIPE
 one 90-degree BEND
 one TEE with run + branch
 one stepped REDUCER

one finite RIGID body
 one RIGID_LINK
 anchor, guide and spring
 nodal force and distributed load
 three B31.3 code points

Required in XY/XZ/YZ/ISO:
 projected coordinates match independent calculations
 source and component IDs match
 click selection matches
 fit bounds match
 accepted command produces expected source revision
 undo reproduces byte-identical initial source
 SVG scene repeat is byte-identical after canonical ordering

13. Anti-drift source guard

- Reject imports from solver, result-recovery or B31.3 packages inside SVG renderer/controller modules.
- Reject stress, SIF, allowable or utilization arithmetic in workspace code.
- Reject direct source mutation from pointermove.
- Reject DOM IDs or SVG path data in engineering hashes.
- Reject localeCompare() in canonical ordering.
- Reject Math.random(), Date.now() and random UUIDs in engineering commands or scene identity.
- Reject runtime network imports from XML_Compare_Uutilities or GitHub Pages.
- Reject Topology-specific source schemas in LFEA production contracts.
- Reject component flattening from BEND/TEE/REDUCER/RIGID to PIPE.
- Reject an editable adapter without a revision-checked command gateway.

```
const forbiddenRules = [
  {
    code: 'SVG_NUMERICAL_AUTHORITY_LEAK',
    root: 'src/workspace/lfea-svg',
    pattern: /calculateStress|allowableStress|utilization\s*=\|factorize|assembleGlobal/u,
  },
  {
    code: 'SVG_POINTER_SOURCE_MUTATION',
    root: 'src/workspace/lfea-svg',
    pattern: /pointermove[\s\S]{0,600}(source|model)\s*\./u,
  },
  {
    code: 'REMOTE_RUNTIME_IMPORT',
    root: 'src',
    pattern: /reallaksh19\.github\.io|raw\.githubusercontent\.com/u,
  },
  {
    code: 'LOCALE_CANONICAL_ORDER',
    root: 'src/workspace/lfea-svg',
    pattern: /localeCompare\s*\(/u,
  }
];
```

14. Exact release gate

```
npm ci
npm run check:lfea-core
npm run check:lfea-workbench
npm run check:lfea-svg-contract
npm run check:lfea-svg-parity
npm run check:lfea-svg-editor
npm run check:lfea-svg-components
npm run check:lfea-svg-anti-drift
npm run check:lfea-svg-performance
npm run check:e2e
npm run syntax:strict
npm run check:imports
npm run build
npm run gate
git diff --check
test -z "$(git status --short)"
```

14.1 Final acceptance

- The LFEA SVG surface is usable without the Topology Validator site being available.
- Source lineage to the reference implementation is recorded and testable.
- All edits pass through revision-checked commands and existing LFEA component authorities.

LFEA SVG Adoption Procedure | Topology Validator Engineering SVG

- SVG preview, viewport and result display remain non-numerical.
- Pipe, bend, tee, reducer, rigid, support, load and code-point workflows are complete.
- Projection, component, source immutability, accessibility and performance tests pass at the exact PR head.

References

- R1.** XML_Compare_Uilities main reference commit 126df8acc370d22540cb129dce789ea04773ebaf.
- R2.** Topology SVG editor model, controller and overlay in tabs/topology-trace-validator.
- R3.** Shared Engineering SVG adapter, command gateway and snap-provider contracts in core/engineering-svg.
- R4.** MDN SVG introduction and DOM/vector model. — https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorials/SVG_from_scratch/Introduction
- R5.** Advanced_Analysis target baseline ea33e6aacc6a3328b648468dbd6a534bc9a4c867.